

CS 473, Spring 2023

Midterm Exam 1

Name:	
NetID:	

- Date/Time: Feb 20, 2023 (Monday), 7:00pm–9:00pm
- You may bring **one double-sided 8.5" × 11" cheat sheet**, with your name and NetID written on the upper right corner. (Two single-sided sheets are okay.) The sheet should be **handwritten** by you (not photocopied or printed).
- Except for your writing implements, your cheat sheet, and your university ID, please put everything away for the duration of the exam. In particular, please turn off and put away *all electronic devices* other than those that are medically necessary.
- This exam booklet is **double-sided!** There are 4 questions (some of which may have multiple parts). The total number of points is 100.
- **At the top of *every* page, please write your name.** (We will scan every page in your exam booklet into Gradescope.)
- All answers should be placed in the spaces given. If you need more space to complete an answer, you may use the blank pages at the end, but please clearly indicate where we should look.
- You may use the blank pages at the end for scratch work.
- Submit your cheat sheet (and all scratch papers) along with your exam booklet when you are finished.
 - If you are NOT using a cheat sheet, please check this box. ☐
- You are reminded about the course's, the department's, and the university's academic integrity policies (the penalty for cheating is severe).
- All exams are strictly confidential for at least 24 hours, or until all conflict exams have been taken. Do not discuss your exam with anyone, either in person or online.

1. [25 PTS] *Short questions.*

(a) [5 PTS] Solve the following recurrence (i.e., give tight Θ bound):

$$T(n) = \begin{cases} 374T(n/473) + n^{3/2} & \text{if } n \geq 473 \\ 2023 & \text{else} \end{cases}$$

Very briefly justify your answer (detailed proofs are not necessary).

(b) [5 PTS] True or False: Karatsuba's divide-and-conquer algorithm is the fastest known algorithm for solving the convolution problem. Justify your answer.

(c) [5 PTS] Consider the “max-plus matrix multiplication” problem: given matrices $A = (a_{ij})_{i,j \in \{1, \dots, n\}}$ and $B = (b_{ij})_{i,j \in \{1, \dots, n\}}$, compute $c_{ij} = \max_{k=1}^n (a_{ik} + b_{kj})$. Can Strassen's divide-and-conquer algorithm be modified to solve this problem in $O(n^{2.81})$ time? Explain why or why not.

- (d) [5 PTS] Consider n points $p_1, \dots, p_n$ on the unit circle arranged in counterclockwise order. Let $d(i, j)$ denote the Euclidean distance between p_i and p_j . Give a short proof that d satisfies the Monge property, i.e., if $i \leq i' \leq j \leq j'$, then $d(i, j) + d(i', j') \geq d(i, j') + d(i', j)$.

- (e) [5 PTS] Analyze the space usage of the following pseudocode:

```
foo( $A[1, \dots, n]$ ):  
  1. if  $n = 1$  return  
  2. create a new array  $B$  of size  $n$   
  3. foo( $A[1, \dots, n/2]$ )  
  4. foo( $A[n/2 + 1, \dots, n]$ )  
  5. foo( $A[n/4 + 1, \dots, 3n/4]$ )
```

2. [25 PTS]

- (a) [19 PTS] Given n polynomials $p_1(x), p_2(x), \dots, p_n(x)$ where each polynomial has degree at most m , we want to compute their product

$$p_1(x)p_2(x) \cdots p_n(x).$$

Describe an efficient divide-and-conquer algorithm to solve this problem, and analyze its running time as a function of n and m . (You may use FFT as a subroutine, and you may assume that arithmetic operations on numbers take constant time each. Your algorithm should work regardless of whether n is even or odd.)

Pseudocode:

Run time analysis:

- (b) [6 PTS] Given a set S of n numbers where each number is an integer between 1 and U , and given a integer $W \leq nU$, we want to decide whether there exists a subset of numbers from S that sums to exactly W . (Note that each number may be used *at most once*.) Use (a) to solve this problem.¹ State the running time as a function of n and U . (Note: there is a dynamic programming algorithm with run time $O(nW)$, but your algorithm should be faster when U is much smaller than W .)

¹Generally, in multi-part questions, you can still do part (b) even if you are unable to do part (a), by assuming that the result in part (a).

3. [25 PTS] Define $x \ominus y = |x - y|$.

We are given a sequence of n integers $\langle a_1, \dots, a_n \rangle$ and an integer b , where $0 \leq a_i \leq U$ and $0 \leq b \leq U$. We want to determine whether it is possible to add parentheses to the expression $a_1 \ominus a_2 \ominus \dots \ominus a_n$ so that the expression evaluates to exactly b .

For example: for the sequence $\langle 3, 5, 6, 9, 1 \rangle$ and $b = 6$, the answer is yes and a solution is $(3 \ominus (5 \ominus 6)) \ominus (9 \ominus 1) = 6$.

To solve this problem using dynamic programming, define the following subproblems: for $1 \leq i \leq j \leq n$ and $x \in \{0, \dots, U\}$, let $C(i, j, x)$ be true iff it is possible to add parentheses so that the expression $a_i \ominus \dots \ominus a_j$ evaluates to x . It is not difficult to see that the following recursive formula holds (omitting base cases which you can fill in):

$$C(i, j, x) = \bigvee_{k=i}^{j-1} \bigvee_{y=x}^U (C(i, k, y) \wedge C(k+1, j, y-x)) \vee \bigvee_{k=i}^{j-1} \bigvee_{y=0}^{U-x} (C(i, k, y) \wedge C(k+1, j, y+x)).$$

Using this formula (which you do not have to prove), first write pseudocode to determine whether it is possible for the input sequence $\langle a_1, \dots, a_n \rangle$ and integer b ; this part should be done using for-loops. Then write pseudocode to output a solution itself (i.e., the parenthesized expression that evaluates to b), when the answer is yes. Analyze the running time and space as a function of n and U .

Pseudocode, first for determining whether the answer is yes or no:

Write Your Name: _____

7

Pseudocode, to output the parenthesized expression when the answer is yes:

Run time and space analysis:

4. [25 PTS] Given a weighted DAG $G = (V, E)$ with n vertices, we want to find 2 vertex-disjoint paths, maximizing the total weight of the paths. (Each vertex can belong to at most one of the two paths; not all vertices need to be covered.) Describe a polynomial-time dynamic programming algorithm to solve this problem.

You may assume that we are given an ordering of the n vertices $v_1, \dots, v_n$ of V , so that all edges $(v_i, v_j) \in E$ have $i < j$ (i.e., we are given a topological sort). Use $w(v_i, v_j)$ to denote the weight of the edge (v_i, v_j) .

Include the following steps: (a) first define your subproblems precisely, (b) then derive the recursive formula (including base cases) with brief justifications, (c) specify the evaluation order, and (d) analyze the running time and space as a function of n . *For this problem, you do not need to provide pseudocode, and you do not need to describe how to output the optimal paths, just the total weight.*

(Hint/suggestion: one possible approach is to work with suffixes of the sequence $v_1, \dots, v_n$, and define a class of subproblems with three parameters (i.e., a 3-dimensional table).)

Definition of subproblems (and indicate which subproblem corresponds to the original problem):

Recursive formula, including base cases and brief justification:

Write Your Name: _____

Evaluation order:

Run time and space analysis:

Write Your Name: _____

10

(Extra space)

Write Your Name: _____

(Extra space)

Write Your Name: _____

12

(Extra space)

Write Your Name: _____

(Extra space)

Write Your Name: _____

(Extra space)